



CENG 5410

Advanced Computer Architecture

Lecture 05: Multiple Issue

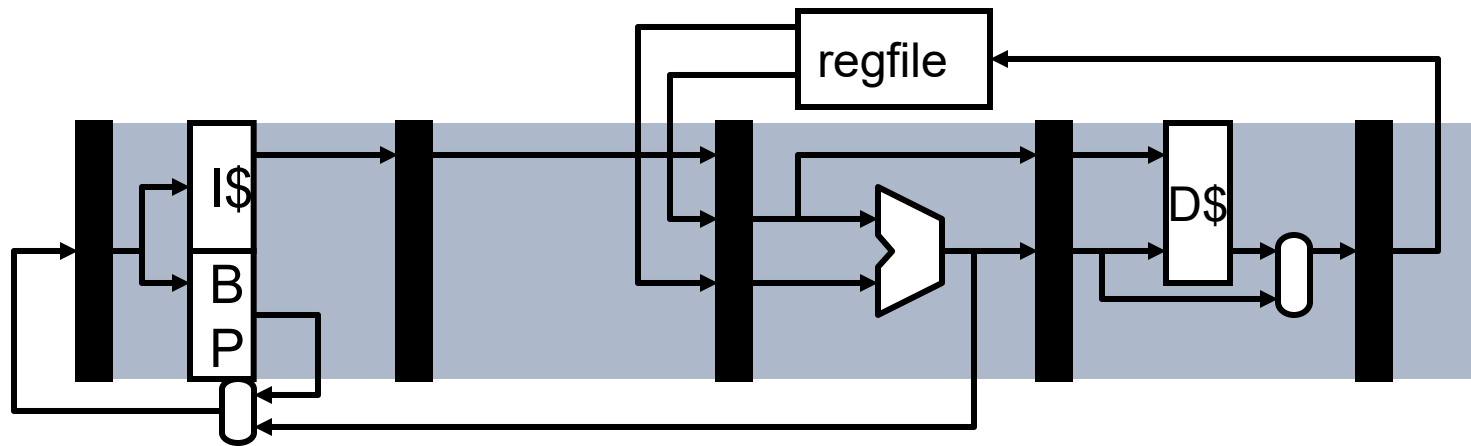
Zhengrong Wang
CSE Department, CUHK
zhengrongwang@cuhk.edu.hk



Scalar Pipeline and the Flynn Bottleneck

- **Scalar pipelines**

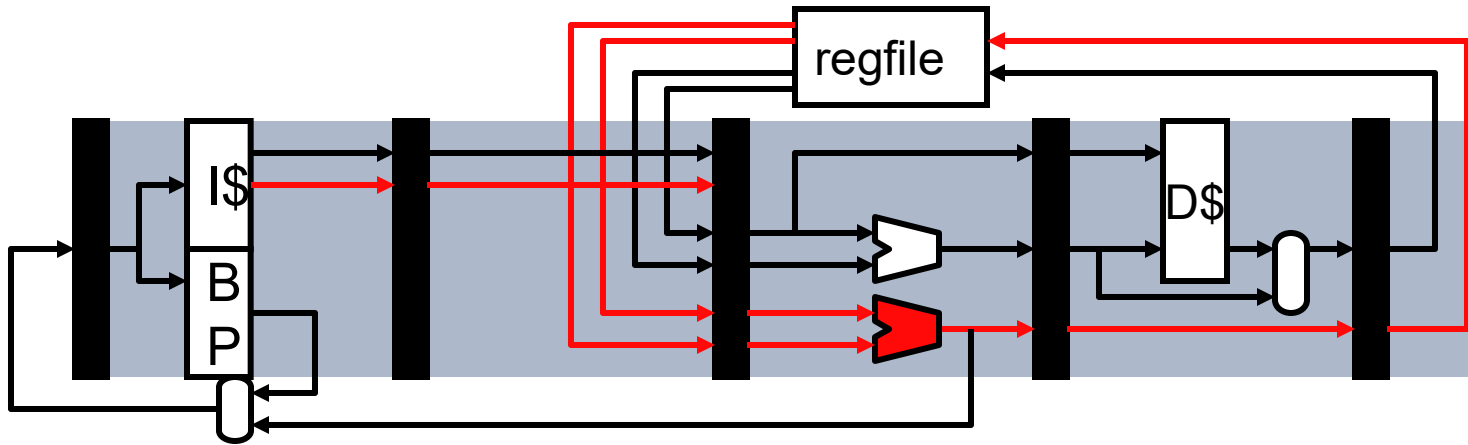
- One instruction per stage
- Performance limit (aka “Flynn Bottleneck”) is $CPI = IPC = 1$
- Limit is never even achieved (hazards)
- Diminishing returns from “super-pipelining” (hazards + overhead)





Superscalar / Multiple-Issue Pipeline

- Overcome this limit using **multiple issue**
 - Also sometimes called **superscalar**
 - Two instructions per stage at once, or three, or four, or eight...
 - “Instruction-Level Parallelism (ILP)” [Fisher]





Superscalar Execution

Single-issue

	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1+0] → r2	F	D	X	M	W							
ld [r1+4] → r3		F	D	X	M	W						
ld [r1+8] → r4			F	D	X	M	W					
ld [r1+12] → r5				F	D	X	M	W				
add r2, r3 → r6					F	D	X	M	W			
add r4, r6 → r7						F	D	X	M	W		
add r5, r7 → r8							F	D	X	M	W	
ld [r8] → r9								F	D	X	M	W

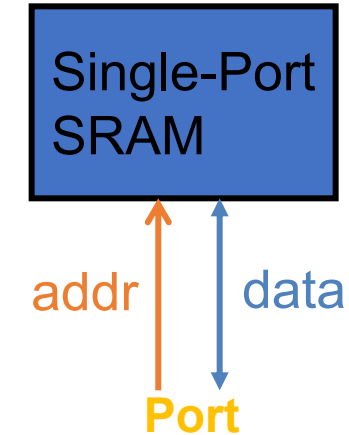
Dual-issue

	1	2	3	4	5	6	7	8	9	10	11
ld [r1+0] → r2	F	D	X	M	W						
ld [r1+4] → r3	F	D	X	M	W						
ld [r1+8] → r4		F	D	X	M	W					
ld [r1+12] → r5		F	D	X	M	W					
add r2, r3 → r6			F	D	X	M	W				
add r4, r6 → r7			F	D	d*	X	M	W			
add r5, r7 → r8				F	D	d*	X	M	W		
ld [r8] → r9				F	s*	D	d*	X	M	W	

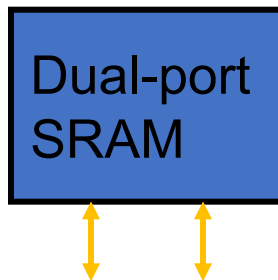


Background: SRAM memories

- SRAMs: Static Random Access Memory
 - Used for most on-chip memories
- Port: SRAM I/O – pair of Address/Data lines
 - Port flavors: Read, Write, Read/write (more expensive)
 - P: Number of ports -- energy $\propto P$, area $\propto P^2$ (yikes)
- Options for parallel memory access:



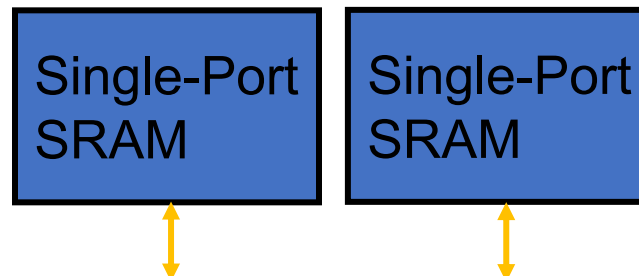
True Multiport



Expensive!

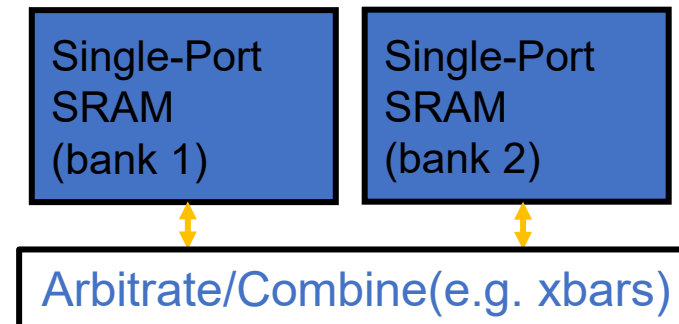
(Reg files)

Replication



Read bandwidth doubles
Writes go to both, so write bandwidth not improved.
Area Overhead!

Banked Memory



Bank conflicts cause stalls (reqs to same bank)

(Caches)



Superscalar Challenges - Front End

- **Wide instruction fetch**
 - Modest: need multiple instructions per cycle
 - *Aggressive: predict multiple branches, trace cache*
- **Wide instruction decode**
 - Replicate decoders (okay for RISC ISA)
- **Wide instruction issue**
 - Determine when instructions can proceed in parallel
 - More complex stall logic - order N^2 for N -wide machine
- **Wide register read**
 - One port for each register read (quadratic complexity in h/w)
 - Example, 4-wide superscalar → ≥ 8 read ports



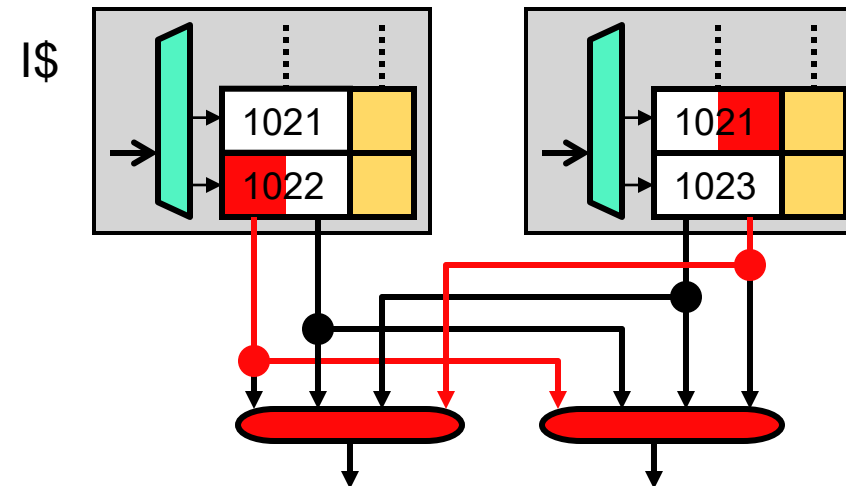
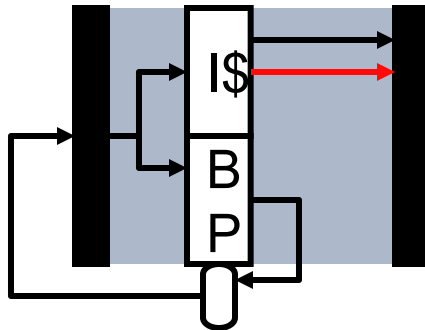
Superscalar Challenges - Back End

- **Wide instruction execution**
 - Replicate arithmetic units
 - Multiple cache ports (if two loads/stores at the same time)
- **Wide instruction register writeback**
 - One write port per instruction that writes a register
 - Example, 4-wide superscalar $\rightarrow \geq 4$ write ports
- **Wide bypass paths**
 - More possible sources for data values
 - Order $(N^2 * P)$ for N -wide machine with execute pipeline depth P
- **Fundamental challenge:**
 - Amount of ILP (instruction-level parallelism) in the program
 - Compiler must schedule code and extract parallelism



Wide Fetch - Sequential Instructions

- What is involved in fetching multiple instructions per cycle?
- In same cache block? → no problem
 - Favors larger block size (independent of hit rate)
- Could compilers align basic blocks to I\$ lines? (pad with **nops**)
 - Reduces effective I\$ capacity
 - + Increases fetch bandwidth utilization (more important)
- In multiple blocks? → Fetch block A and A+1 in parallel
 - Banked I\$ + **combining network**
 - May add latency (add pipeline stages to avoid slowing down clock)



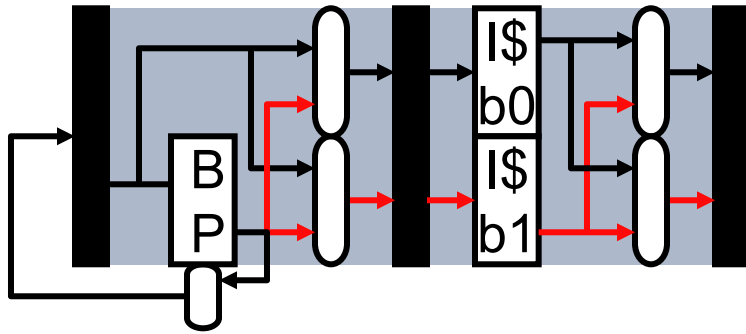


Wide Fetch - Non-sequential

- Two related questions
 - How many branches predicted per cycle?
 - Can we fetch from multiple taken branches per cycle?
- Simplest, most common organization: “1” and “No”
 - One prediction, discard post-branch insns if prediction is “Taken”
 - Lowers effective fetch width and IPC
 - Consider a 10-instruction loop body with an 8-issue processor
 - Without smarter fetch, ILP is limited to 5 (not 8)
- Compiler can help looping case...
 - Unroll loops, reduce taken branch frequency



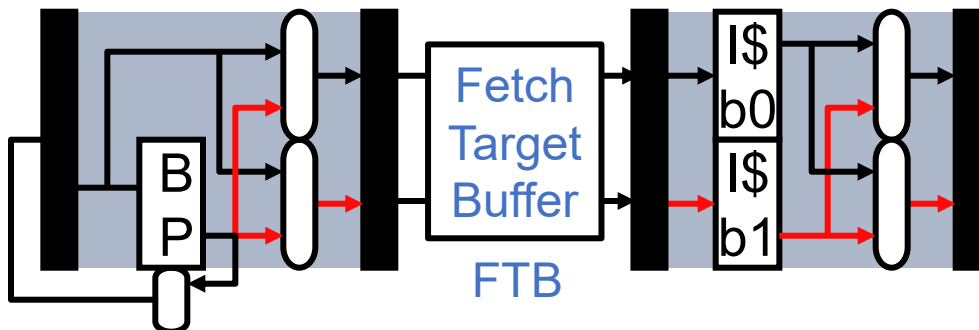
Parallel Non-Sequential Fetch



Tag	Data (insns)
0	addi , beq #4, ld, sub
4	st , call #32, ld, add

Transmit lines 0 and 4 to the next phase

- Allowing “embedded” taken branches is possible
 - Requires parallel branch predictor, multiple I\$ accesses in one cycle
- Can try pipelining branch prediction and fetch
 - Branch prediction stage only needs PC
 - Transmits two PCs to fetch stage, PC and target PC
 - Elongates pipeline, increases branch penalty

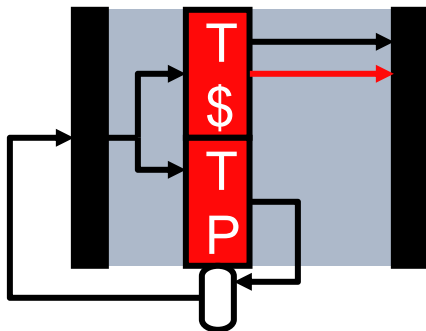


- Decoupled Frontend-Reinman
 - Let branch predictor run ahead even if fetch stalls
 - Fetch directed prefetching (ToT Award): Prefetch from FTB



Trace Cache

- **Trace cache (T\$)** [Peleg+Weiser, Rotenberg+]
 - Overcomes serialization of prediction and fetch by combining them
 - New kind of I\$ that stores **dynamic**, not static, insn sequences
 - Blocks can contain statically non-contiguous insns
 - Tag: PC of first insn + N/T of embedded branches
 - Used in Pentium 4 (actually stores decoded μ ops)
- Coupled with **trace predictor (TP)**
 - Predicts next trace, not next branch



Traditional
ICache

Tag	Data (insns)
0	addi, beq #4, ld, sub
4	st, call #32, ld, add

Trace
Cache

Tag	Data (insns)
0:T	addi, beq #4, st, call #32



Trace Cache Example

- Assume 4-issue processor + single-ported icache
- Traditional instruction cache

Tag	Data (insns)
0	addi, beq #4 , ld, sub
4	st, call #32 , ld, add

0: addi r1, 4, r1
1: beq r1, #4
4: st r1, 4(sp)
5: call #32

1	2
F	D
F	D
f*	F
f*	F

- Trace cache

Tag	Data (insns)
0:T	addi, beq #4, st, call #32

0: addi r1, 4, r1
1: beq r1, #4
4: st r1, 4(sp)
5: call #32

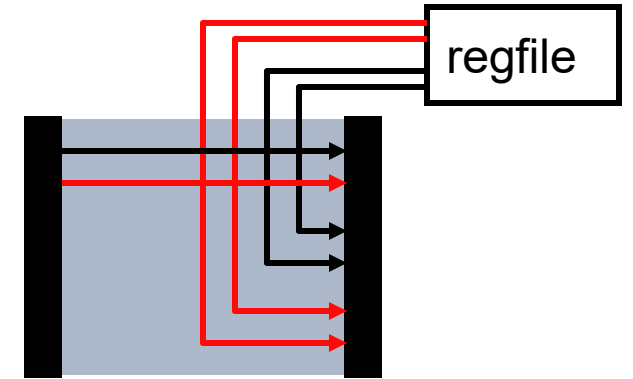
1	2
F	D
F	D
F	D
F	D

- Traces can store pre-decoded dependence information (fun!)
 - Helps fix the N^2 dependence check problem



Wide Decode

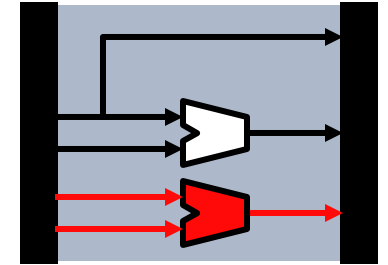
- What is involved in decoding multiple (N) insns per cycle?
- Actually doing the decoding?
 - Easy if fixed length (multiple decoders), doable if variable length
- Reading input registers?
 - **2N register read ports** (latency $\propto$ #ports)
 - + Actually less than 2N, most values come from bypasses
- What about the stall logic? (**quadratic**)
 - N instructions that need checking
 - Each of those are checked against factor of N previous instructions





Wide Execute

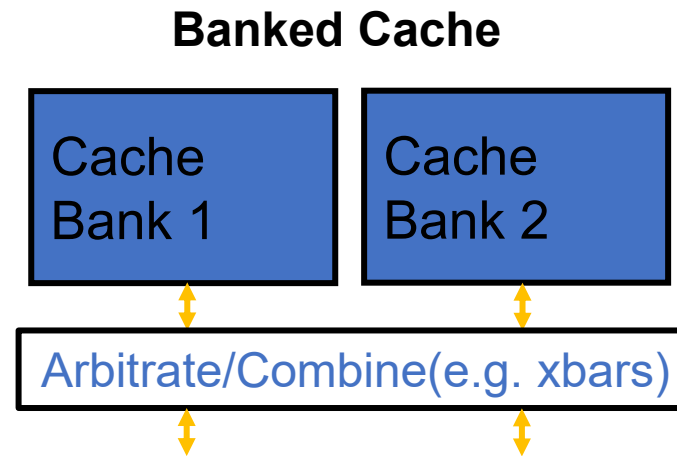
- What is involved in executing multiple (N) insns per cycle?
- Multiple execution units ... N of every kind?
 - N ALUs? OK, ALUs are small
 - N FP dividers? No, FP dividers are huge and **fdiv** is uncommon
 - How many branches per cycle?
 - How many loads/stores per cycle?





Wide Memory Access

- Probably you're going to bank the Cache:

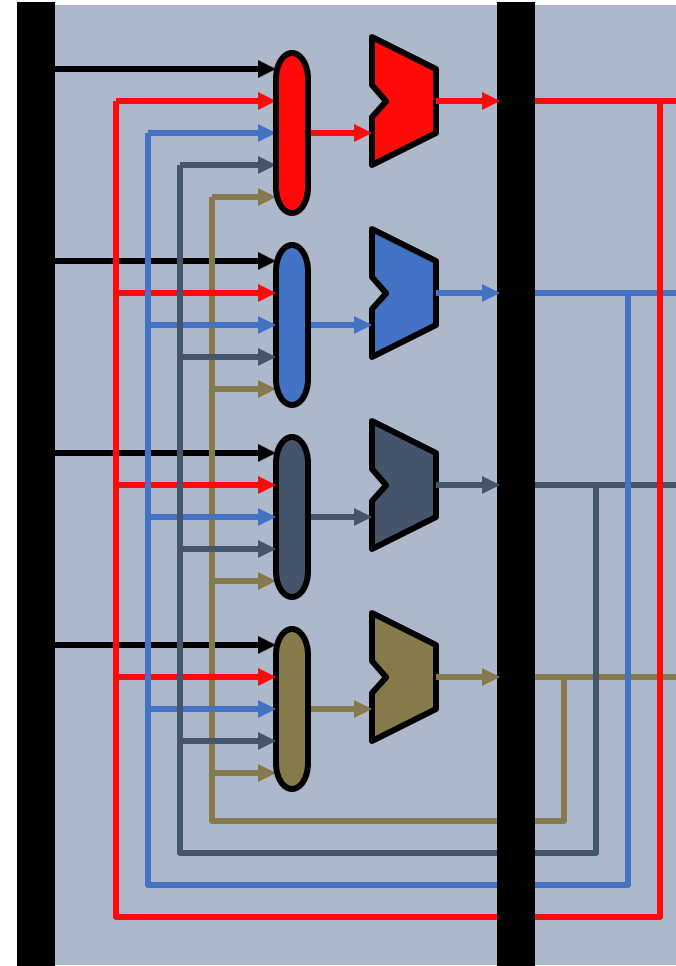


- Funny example: Alpha 21164 replicates L1 data
 - 8KB L1-caches, supports two loads, but only one store



Wide Bypassing ... oh no what happened?

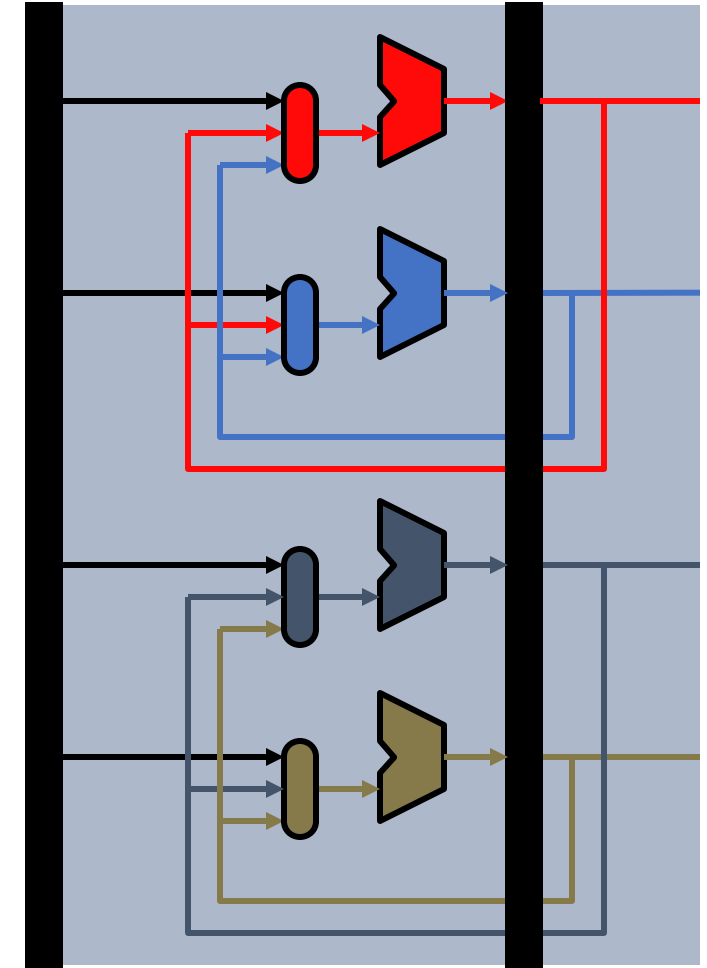
- **All to all communication :** (
 - **N^2 stall and bypass logic**
 - Actually OK
 - 5-bit and 1-bit quantities
 - **N^2 bypass network**
 - 32-bit (or 64-bit) quantities
 - Routing lengthens wires
 - Expensive metal layer crossings
 - $N+1$ input muxes at each ALU input
 - And this is just one bypassing stage!





Clustering

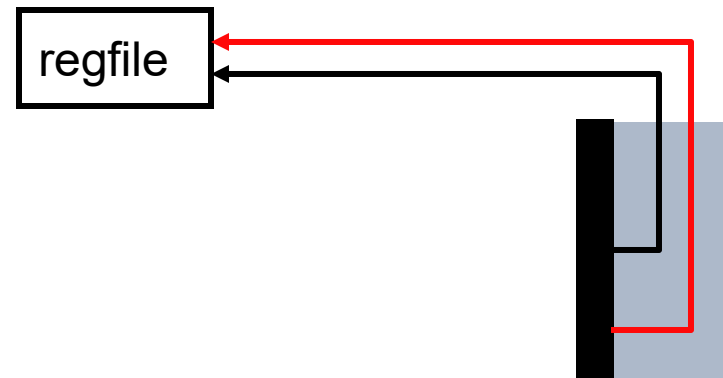
- **Clustering**: mitigates N^2 bypass
 - Group FUs into **K** clusters
 - Full bypassing within a cluster
 - Limited bypassing between clusters
 - With a one cycle delay
 - $(N/K) + 1$ inputs at each mux
 - $(N/K)^2$ bypass paths in each cluster
- **Steering**: key to performance
 - Steer dependent insns to same cluster
 - Statically (compiler) or dynamically
- E.g., Alpha 21264
 - Bypass wouldn't fit into clock cycle
 - 4-wide, 2 clusters, static steering based on instruction type (boring)
 - Replicates register file, too





Wide Writeback

- What is involved in multiple (N) writebacks per cycle?
 - N register file write ports (**expensive** -- latency $\propto$ #ports)
 - Usually less than N, stores and branches don't do writeback
- Multiple exceptions per cycle?
 - No just the oldest one





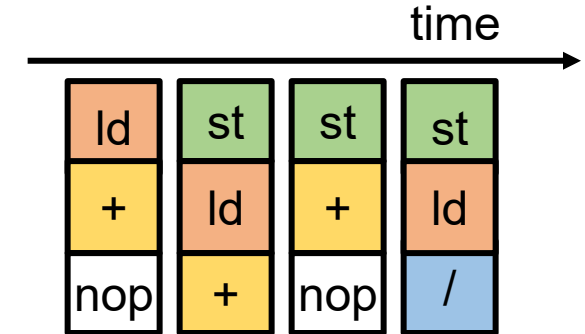
Superscalar Hardware Stinks...

- Hardware-centric multiple issue problems
 - Wide fetch+branch prediction, N^2 bypass, N^2 dependence checks
 - Hardware solutions have been proposed: clustering, trace cache

So lets let the compiler do it!



- Software-centric: **very long insn word (VLIW)**
 - Effectively, a 1-wide pipeline, but unit is an N-insn group
 - Compiler guarantees insns within a VLIW group are independent
 - If no independent insns, slots filled with **nops**
 - Group travels down pipeline as a unit
 - + Wide fetch is simpler now
 - + Dependence Cross-checks within a group un-necessary
 - Still check downstream (further down the pipeline)
 - Typically “slotted”: 1st insn must be ALU, 2nd mem, etc.
 - Bypassing the same, but...
 - + Compiler could schedule for limited bypassing / clustering
 - Introduces some concerns about **compatibility**





Pure and Tainted VLIW

- **Pure VLIW**: no hardware dependence checks at all (not even inter-group)
 - + Very simple and low power hardware
 - Compiler responsible for scheduling stall cycles
 - Requires precise knowledge of pipeline depth and structure
 - Recompile if the hardware changes!!!!
 - Doesn't support caches well
 - Used in some cache-less micro-controllers and signal processors
- **Realistic VLIW**: inter-group checks + stalls
 - Compiler doesn't schedule stall cycles
 - + Precise pipeline depth and latencies not needed, can be changed
 - + Supports caches
 - TransMeta Crusoe
- **Intel's take: Explicitly Parallel Instruction Computing (EPIC)**
 - Implemented as “bundles” with explicit dependence bits
 - Code is compatible with different “bundle” width machines
 - Compiler discovers as much parallelism as it can, hardware does rest
 - E.g., Intel Itanium (IA-64)
 - 128-bit bundles (3 41-bit insns + few extra bits)



Terminology Aside: Schedule and Issue

- **Terminology:**
 - “Static”: Compiler Decides, encoded into program
 - “Dynamic”: Hardware Decides, determined at runtime
- **Dynamic vs Static Issue:**
 - Static: timing predefined by compiler, must guarantee no structure or dependence hazards
 - Dynamic: timing hardware stalls to avoid hazards
- **Schedule:** order in which insns execute
 - Static: ordering of instructions predefined by compiler, compiler reorders for performance
 - Dynamic: ordering of instructions determined by hardware (e.g. dataflow)
- **Schedule/issue combinations**
 - Pure VLIW: static schedule, static issue
 - Superscalar, EPIC: static schedule, dynamic issue



History of VLIW

- Started with “horizontal microcode”
 - Culler-Harrison array processors ('72-'91)
 - Floating Point Systems FPS-120B
- Academic projects
 - Yale ELI-512 [Fisher, '85]
 - Illinois IMPACT [Hwu, '91]
- Commercial attempts
 - Multiflow [Colwell+Fisher, '85] → failed
 - Cydrome [Rau, '85] → failed
 - Motorola/TI embedded processors → successful
 - Intel Itanium [Colwell,Fisher+Rau, '97] → failed slowly
 - Itanium 9700 (Kittson): 2017 – last itanium
 - Transmeta Crusoe [Ditzel, '99] → failed

Very Cool!



IA-64 Instruction Format

- IA-64 template specifies
 - The type of operation for each instruction
 - MFI, M/MI, MI/I, MLI, MIB, MMF, MFB, MMB, MBB, BBB –
 - Intra-bundle relationship,
 - M / MI or MI / I (/ is a “stop” meaning no parallelism) –
 - Inter-bundle relationship
- Most common combinations covered by templates
- Fewer templates -> simpler hardware
- Scales compatibly to future generations
- Future microarchitetures can execute N Bundles per cycle

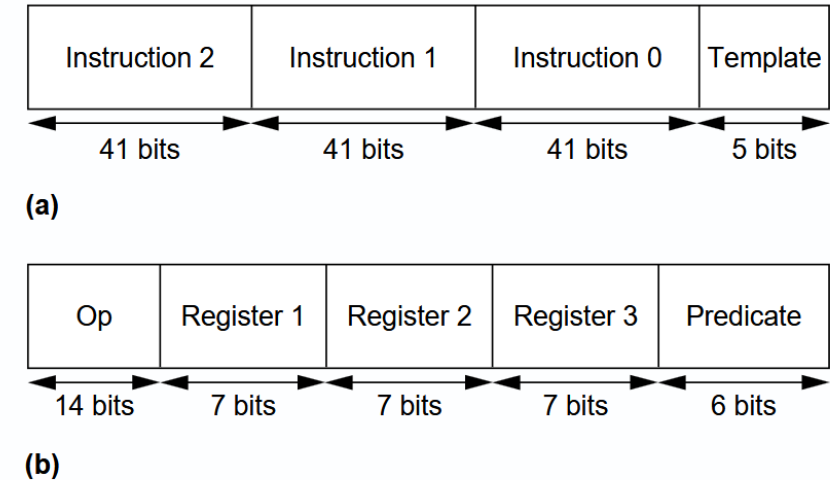


Figure 2. IA-64 bundle (a) and instruction (b) formats.



ILP and Static Scheduling

- No point to having an N-wide pipeline...
- ...if average number of parallel insns per cycle (ILP) $\ll N$
- How can the compiler help extract parallelism?
 - These techniques applicable to regular superscalar
 - These techniques critical for VLIW/EPIC



Code Example: SAXPY

- **SAXPY** (Single-precision A X Plus Y)
 - Linear algebra routine (used in solving systems of equations)
 - Part of early “Livermore Loops” benchmark suite

```
for (i=0;i<N;i++)  
  Z[i]=A*X[i]+Y[i];
```

```
0: ldf X(r1)→f1          // loop  
1: mulf f0,f1→f2         // A in f0  
2: ldf Y(r1)→f3         // X,Y,Z are constant addresses  
3: addf f2,f3→f4  
4: stf f4→Z(r1)  
5: addi r1→4,r1          // i in r1  
6: blt r1→r2,0           // N*4 in r2
```



SAXPY Performance and Utilization

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
ldf x(r1) → f1	F	D	X	M	W															
mulf f0, f1 → f2		F	D	d*	E*	E*	E*	E*	E*	W										
ldf y(r1) → f3			F	p*	D	X	M	W												
addf f2, f3 → f4					F	D	d*	d*	d*	E+	E+	W								
stf f4 → Z(r1)						F	p*	p*	p*	D	X	M	W							
addi r1, 4 → r1										F	D	X	M	W						
blt r1, r2, 0											F	D	X	M	W					
ldf x(r1) → f1												F	D	X	M	W				

- Scalar pipeline

- Full bypassing, 5-cycle E*, 2-cycle E+, branches predicted taken
- Single iteration (7 insns) latency: 16–5 = 11 cycles
- **Performance**: 7 insns / 11 cycles = 0.64 IPC
- **Utilization**: 0.64 actual IPC / 1 peak IPC = 64%



SAXPY Performance and Utilization

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
ldf x(r1) → f1	F	D	X	M	W															
mulf f0, f1 → f2	F	D	d*	d*	E*	E*	E*	E*	E*	W										
ldf y(r1) → f3		F	D	p*	X	M	W													
addf f2, f3 → f4		F	p*	p*	D	d*	d*	d*	d*	E+	E+	W								
stf f4 → z(r1)			F	p*	D	p*	p*	p*	p*	d*	X	M	W							
addi r1, 4 → r1					F	p*	p*	p*	p*	p*	D	X	M	W						
blt r1, r2, 0					F	p*	p*	p*	p*	p*	D	d*	X	M	W					
ldf x(r1) → f1											F	D	X	M	W					

- Dual issue pipeline
- Same + any two insns per cycle + embedded taken branches
 - + **Performance**: 7 insns / 10 cycles = 0.70 IPC
 - **Utilization**: 0.70 actual IPC / 2 peak IPC = 35%
 - More hazards → more stalls (why?)
 - Each stall is more expensive (why?)



Instruction Scheduling

- Idea: place independent insns between slow ops and uses
 - Otherwise, pipeline stalls while waiting for RAW hazards to resolve
 - Have already seen pipeline scheduling
- To schedule we'll need ... **independent insns**
- **Scheduling scope**: code region we are scheduling
 - The bigger the better (more independent insns to choose from)
 - Difficult to schedule across branches... especially loops
- Compiler scheduling (really scope enlarging) techniques
 - Loop unrolling (for loops)
 - Trace scheduling (for non-loop control flow)



Loop Unrolling SAXPY

- Goal: separate dependent insns from one another
- SAXPY problem: not enough flexibility within one iteration
 - Longest chain of insns is 9 cycles
 - Load (1)
 - Forward to multiply (5)
 - Forward to add (2)
 - Forward to store (1)
 - Can't hide a 9-cycle chain using only 7 insns
 - But how about two 9-cycle chains using 14 insns?
- **Loop unrolling**: schedule two or more iterations together
 - Fuse iterations
 - Pipeline schedule to reduce RAW stalls
 - Pipeline schedule introduces WAR violations, rename registers to fix



Unrolling SAXPY I: Fuse Iterations

- Combine two (in general K) iterations of loop
 - Fuse loop control: induction variable (**i**) increment + branch
 - Adjust implicit induction uses

```
ldf X(r1) → f1
mul f0, f1 → f2
ldf Y(r1) → f3
add f2, f3 → f4
stf f4 → Z(r1)
addi r1, 4 → r1
blt r1, r2, 0
```



```
ldf X(r1) → f1
mul f0, f1 → f2
ldf Y(r1) → f3
add f2, f3 → f4
stf f4 → Z(r1)
```

```
ldf X(r1) → f1
mul f0, f1 → f2
ldf Y(r1) → f3
add f2, f3 → f4
stf f4 → Z(r1)
addi r1, 4 → r1
blt r1, r2, 0
```

```
ldf X+4(r1) → f1
mul f0, f1 → f2
ldf Y+4(r1) → f3
add f2, f3 → f4
stf f4 → Z+4(r1)
addi r1, 8 → r1
blt r1, r2, 0
```



Unrolling SAXPY II: Pipeline Schedule

- Pipeline schedule to reduce RAW stalls
 - Have already seen this: pipeline scheduling

```
ldf X(r1) → f1
mulf f0, f1 → f2
ldf Y(r1) → f3
addf f2, f3 → f4
stf f4 → Z(r1)
ldf X+4(r1) → f1
mulf f0, f1 → f2
ldf Y+4(r1) → f3
addf f2, f3 → f4
stf f4 → Z+4(r1)
addi r1, 8 → r1
blt r1, r2, 0
```



```
ldf X(r1) → f1
ldf X+4(r1) → f1
mulf f0, f1 → f2
mulf f0, f1 → f2
ldf Y(r1) → f3
ldf Y+4(r1) → f3
addf f2, f3 → f4
addf f2, f3 → f4
stf f4 → Z(r1)
stf f4 → Z+4(r1)
addi r1, 8 → r1
blt r1, r2, 0
```



Unrolling SAXPY III: Rename Registers

- Pipeline scheduling causes WAR violations
 - Rename registers to correct

```
ldf X(r1) → f1
ldf X+4(r1) → f1
mulf f0, f1 → f2
mulf f0, f1 → f2
ldf Y(r1) → f3
ldf Y+4(r1) → f3
addf f2, f3 → f4
addf f2, f3 → f4
stf f4 → Z(r1)
stf f4 → Z+4(r1)
addi r1, 8 → r1
blt r1, r2, 0
```



```
ldf X(r1) → f1
ldf X+4(r1) → f5
mulf f0, f1 → f2
mulf f0, f5 → f6
ldf Y(r1) → f3
ldf Y+4(r1) → f7
addf f2, f3 → f4
addf f6, f7 → f8
stf f4 → Z(r1)
stf f8 → Z+4(r1)
addi r1, 8 → r1
blt r1, r2, 0
```

Aside: Not usually done in this order! :)
Compiler uses SSA representation, then register allocation later!



Unrolled SAXPY Performance/Utilization

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
ldf X(r1)→f1	F	D	X	M	W															
ldf X+4(r1) →f5		F	D	X	M	W														
mulf f0,f1→f2			F	D	E*	E*	E*	E*	E*	W										
mulf f0,f5→f6				F	D	E*	E*	E*	E*	E*	W									
ldf Y(r1) →f3				F		D	X	M	W											
ldf Y+4(r1) →f7						F	D	X	M	s*	s*	W								
addf f2,f3→f4							F	D	d*	E+	E+	s*	W							
addf f6,f7→f8								F	p*	D	E+	p*	E+	W						
stf f4→Z(r1)									F	D	X	M	W							
stf f8→Z+4(r1)										F	D	X	M	W						
addi r1,8→r1											F	D	X	M	W					
blt r1,r2,0												F	D	X	M	W				
ldf X(r1) →f1													F	D	X	M	W			

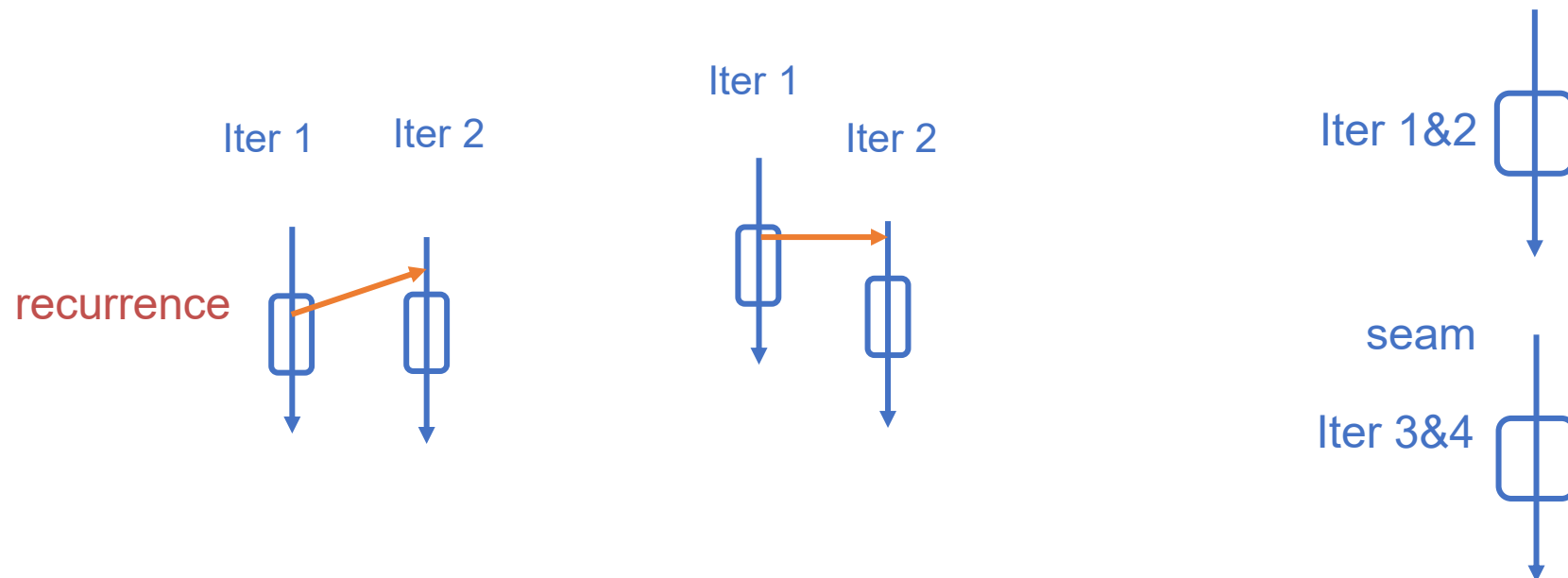
No propagation?
Different pipelines

- +Performance: 12 insn / 13 cycles = 0.92 IPC
- +Utilization: 0.92 actual IPC / 1 peak IPC = 92%
- +**Speedup**: (2 * 11 cycles) / 13 cycles = 1.69



Loop Unrolling Shortcomings

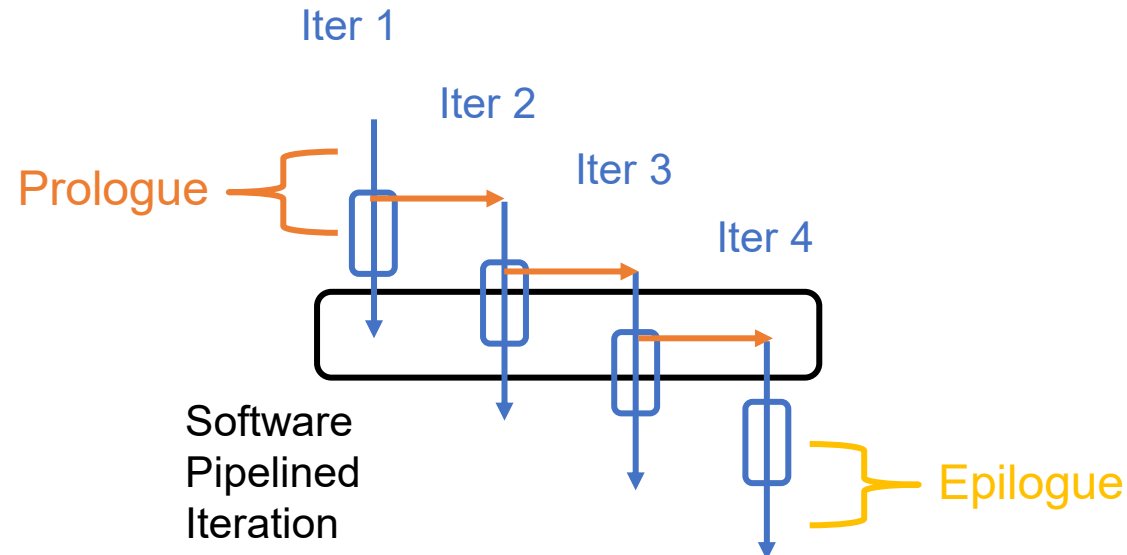
- Static code growth more I\$ misses
 - Use profiling to focus on cases that matter?
- Poor scheduling along “seams” of unrolled copies
- Need more registers to resolve WAR hazards (**more live vars.**)
- **Recurrences reduce the effectiveness of unrolling+scheduling** (inter-iteration dependences)





Alternative: Software Pipelining

- Observation:
 - Even in the presence of dependences, can get ILP by taking instructions from different iterations
 - Take disjoint instructions from iterations -> don't need multiple copies for different iterations
- Software pipelining: reorganizes loops so that each iteration is made from instructions chosen from different iterations of the original loop





Software Pipelining Example

```
ldf X(r1)→f1  
mulf f0,f1→f2  
ldf Y(r1)→f3  
addf f2,f3→f4  
stf f4→Z(r1)  
addi r1,4→r1  
blt r1,r2,0
```

```
ldf X(r1)→f1  
mulf f0,f1→f2  
ldf Y(r1)→f3  
addf f2,f3→f4  
stf f4→Z(r1)  
addi r1,4→r1  
blt r1,r2,0
```

```
ldf X(r1)→f1  
mulf f0,f1→f2  
ldf Y(r1)→f3  
addf f2,f3→f4  
stf f4→Z(r1)  
addi r1,4→r1  
blt r1,r2,0
```

```
ldf X(r1)→f1  
mulf f0,f1→f2  
ldf Y(r1)→f3  
addf f2,f3→f4  
stf f4→Z(r1)  
addi r1,4→r1  
blt r1,r2,0
```

```
stf f4→Z(r1)  
addf f2,f3→f4  
mulf f0,f1→f2  
ldf Y+8(r1)→f3  
ldf X+12(r1)→f1  
addi r1,4→r1  
blt r1,r2,0
```

MMM Perfect for VLIW!



Beyond Scheduling Loops

- Problem: not everything is a loop
 - How to create large scheduling scopes from non-loop code?
- Idea: **trace scheduling** [Ellis, '85]
 - Find common paths in program (profile)
 - Realign basic blocks to form straight-line “traces”
 - **Basic-block**: single-entry, single-exit insn sequence
 - **Trace**: fused basic block sequence
 - Schedule insns within a trace
 - This is the easy part
 - Create **fixup code** outside trace
 - In case implicit trace path doesn't equal actual path
 - Nasty
 - Good scheduling needs **ISA support for software speculation**



Aside: Profiling

- **Profile**: statistical information about program tendencies
 - Software's answer to everything
 - Collected from previous program runs (different inputs)
 - ± Works OK depending on information
 - Memory latencies (cache misses)
 - + Identities of frequently missing loads stable across inputs
 - But are tied to cache configuration
 - Memory dependences
 - + Stable across inputs
 - But exploiting this information is hard (need hw help)
 - Branch outcomes
 - Not always stable across inputs
 - More difficult to use, need to run program and then re-compile



Trace Scheduling Example

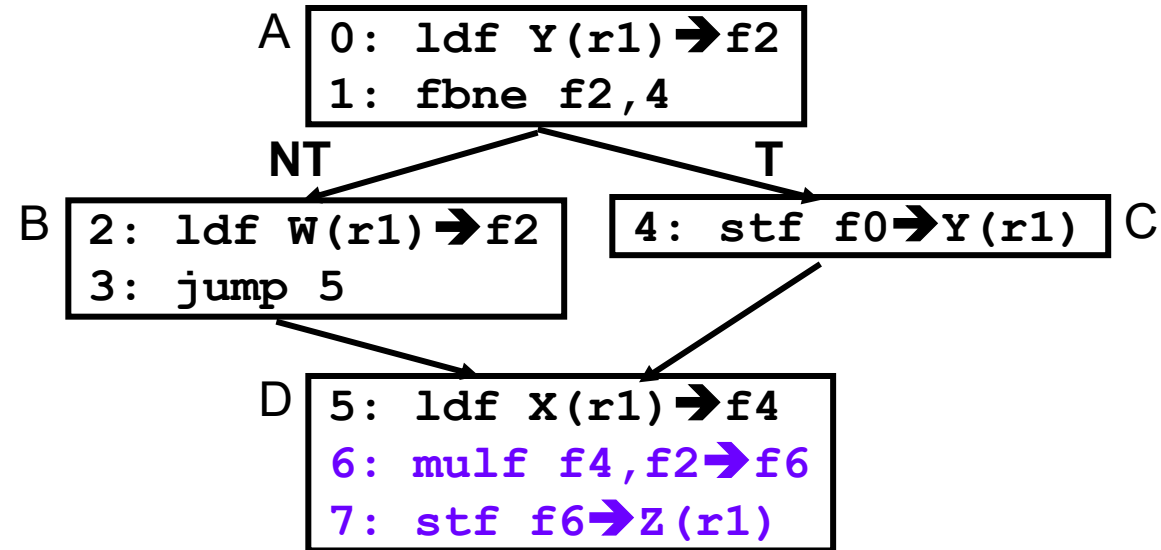
Source code

```
A = Y[i];  
if (A == 0)  
    A = W[i];  
else  
    Y[i] = 0;  
    Z[i] = A*X[i];
```

Machine code

```
0: ldf Y(r1) → f2  
1: fbne f2 → 4  
2: ldf W(r1) f2  
3: jump 5  
4: stf f0 → Y(r1)  
5: ldf X(r1) → f4  
6: mul f4, f2 → f6  
7: stf f6 → Z(r1)
```

4 basic blocks: A,B,C,D

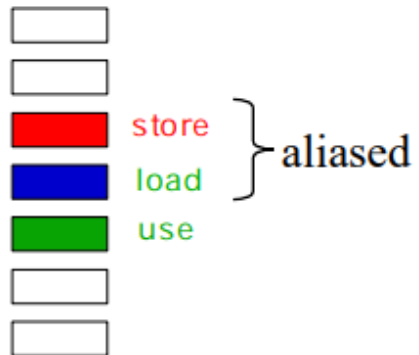


- Problem: separate #6 (3 cycles) from #7
 - 5,6 are dependent
- Want to move 5,6 above 1
- Optimize trace depending on branch direction:
 - Else path: What if the store Y and load X alias?
 - If path: dependence on reg for A

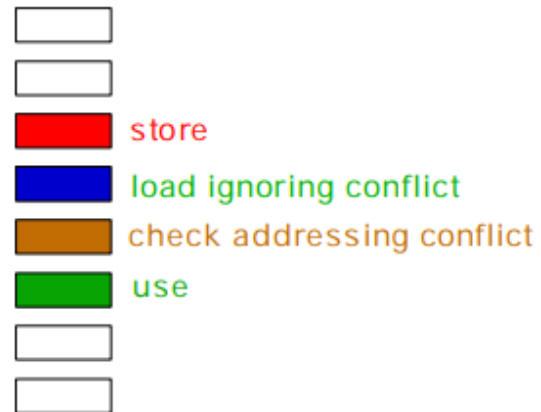


Data Speculation

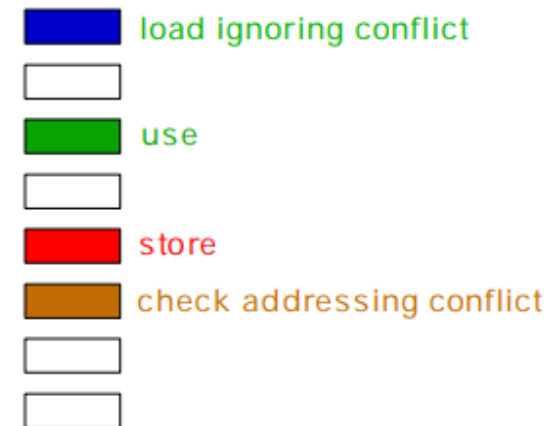
Original



Transform

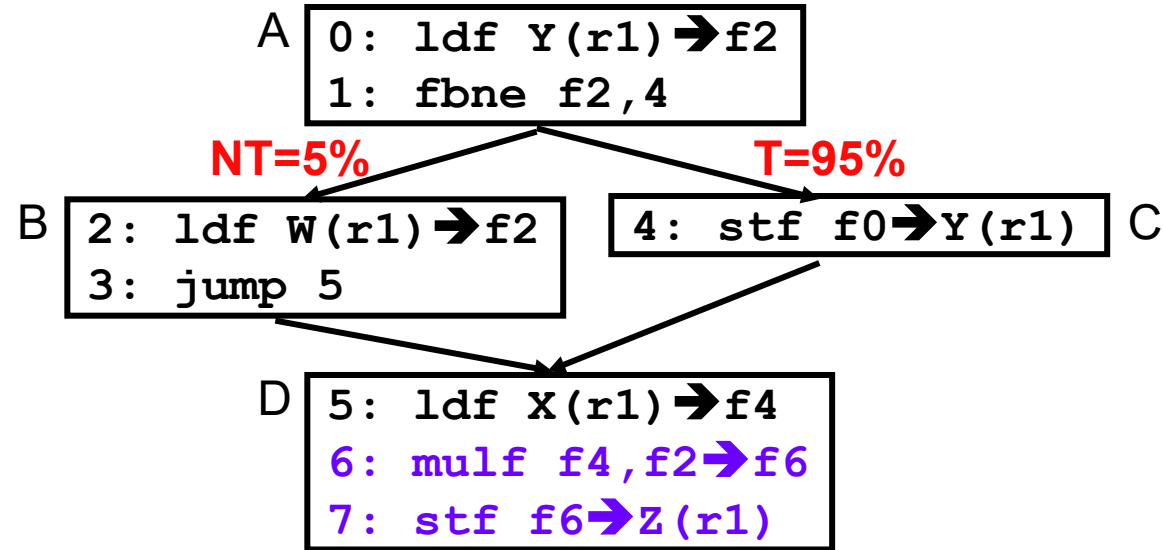


Reschedule





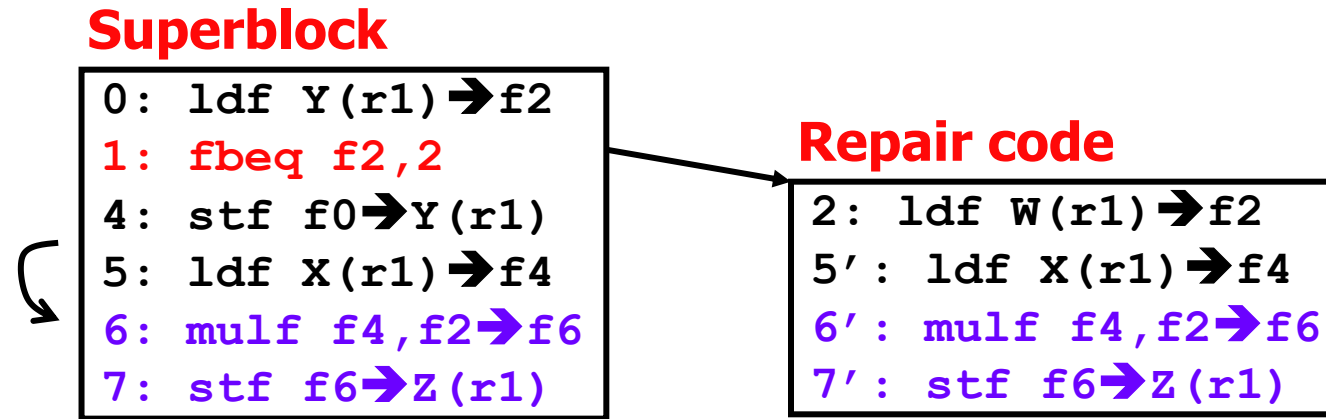
Superblocks



- First trace scheduling construct: **superblock**
 - Use when branch is highly biased
 - Fuse blocks from most frequent path: A,C,D
 - Schedule
 - Create **repair code** in case real path was A,B,D



Superblock and Repair Code



- What did we do?
- Change sense (test) of branch 1
 - Original taken target now fall-thru
 - Created repair block
 - May need to duplicate some code (here basic-block D)
- Haven't actually scheduled superblock yet



Superblocks Scheduling I

Superblock

```
0: ldf Y(r1) → f2  
1: fbeq f2, 2  
5: ldf X(r1) → f4  
6: mulf f4, f2 → f6  
4: stf f0 → Y(r1)  
7: stf f6 → Z(r1)
```

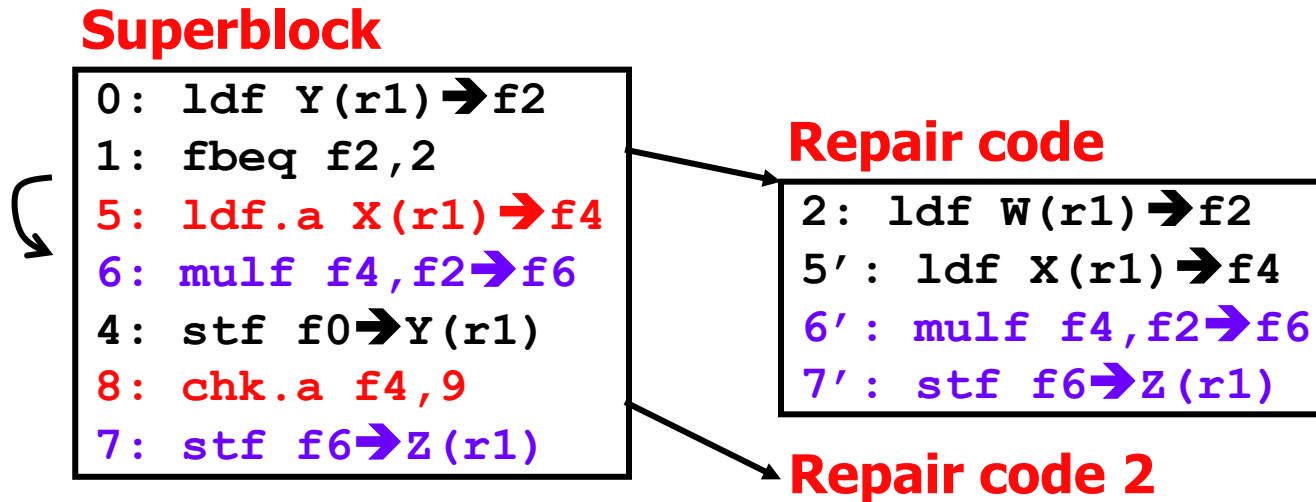
Repair code

```
2: ldf W(r1) → f2  
5': ldf X(r1) → f4  
6': mulf f4, f2 → f6  
7': stf f6 → Z(r1)
```

- First scheduling move: move insns 5 and 6 above insn 4
 - Hmm: moved load (5) above store (4)
 - We can tell this is OK, but can the compiler
 - If yes, fine
 - Otherwise, need to do something



ISA Support for Load/Store Speculation



- IA-64: change insn 5 to **advanced load ldf.a**
 - “Advanced” means advanced past some unknown store
 - Processor stores [address, reg] of advanced loads in table
 - Memory Conflict Buffer (MCB), Advanced Load Alias Table (ALAT)
 - Later stores search ALAT: matching address → invalidate ALAT entry
 - Insert check insn **chk.a** to make sure ALAT entry still valid
 - If not, jump to some more repair code (arghhh...)



Superblock Scheduling II

Superblock

```
0: ldf Y(r1) → f2  
5: ldf.a X(r1) → f4  
6: mulf f4, f2 → f6  
1: fbeq f2, 2  
4: stf f0 → Y(r1)  
8: chk.a f4, 9  
7: stf f6 → Z(r1)
```

Repair code

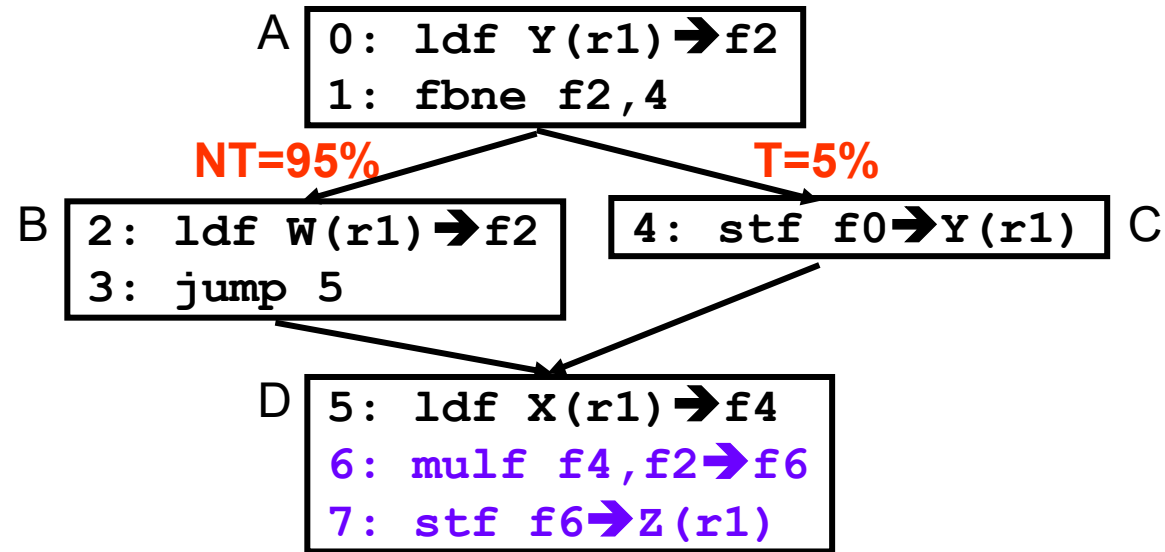
```
2: ldf W(r1) → f2  
5': ldf X(r1) → f4  
6': mulf f4, f2 → f6  
7': stf f6 → Z(r1)
```

Repair code 2

- Second scheduling move: move insn 5 and 6 above insn 1
 - That's OK, load did not depend on branch...
 - And would have executed anyway
 - But it's a little more expensive since they're re-executed on a mistake in the first repair code :)
- Scheduling non-move: don't move insn 4 above insn 1
 - Why? Hard (but possible) to undo a store in repair code
- **Success:** scheduled 3 insns between 6 and 7



What If...



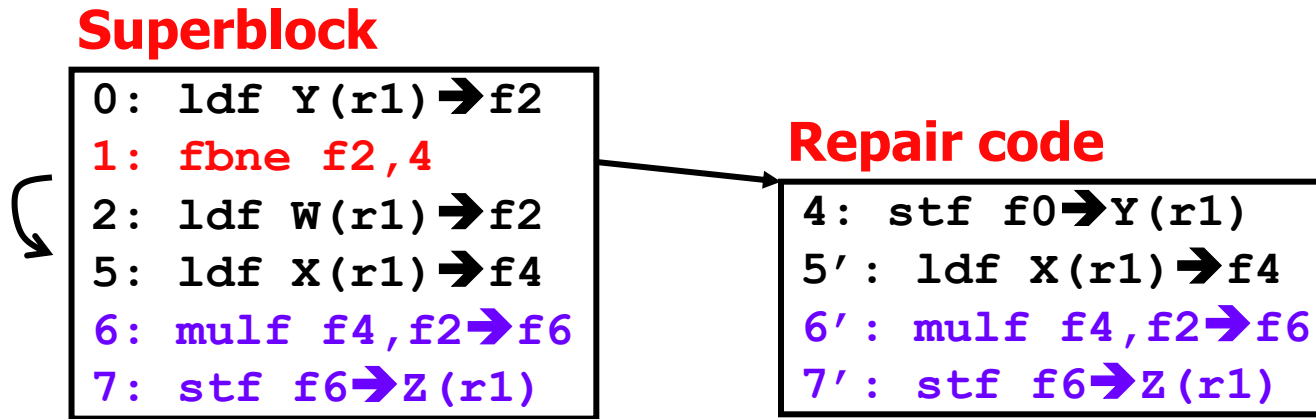
- ... branch 1 had the opposite bias?

Want to move 2,5,6 above branch!

Problem: what if instruction 2 has a fault?



The Other Superblock and Repair Code



- Notice
 - Branch 1 sense (test) unchanged
 - Original taken target now in repair code



Superblock Scheduling III

Superblock

```
0: ldf Y(r1) → f2
2: ldf W(r1) → f8
5: ldf X(r1) → f4
6: mul f4, f8 → f6
1: fbne f2, 4
7: stf f6 → Z(r1)
```

Repair code

```
4: stf f0 → Y(r1)
6': mul f4, f2 → f6
7': stf f6 → Z(r1)
```

- First scheduling move: move insns 2, 5, and 6 above insn 1
 - Rename **f2** to **f8** to avoid WAR violation (for the load of W)
 - Notice, can remove copy of insn 5 from repair code
 - Is this scheduling move legal?
 - From a store standpoint, yes
 - What about from a fault standpoint? What if insn 2 faults?



ISA Support for Load-Branch Speculation

Superblock

```
0: ldf Y(r1) → f2  
2: ldf.s W(r1) → f8  
5: ldf X(r1) → f4  
6: mul f4, f8 → f6  
1: fbne f2, 4  
8: chk.s f8  
7: stf f6 → Z(r1)
```

Repair code

```
4: stf f0 → Y(r1)  
6': mul f4, f2 → f6  
7': stf f6 → Z(r1)
```

Repair code 2

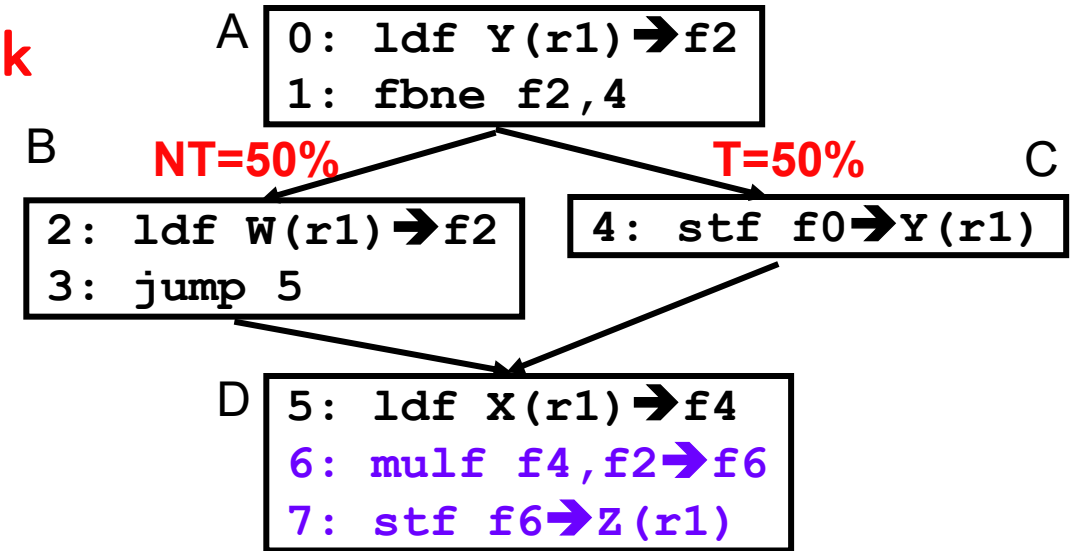
- IA-64: change insn 2 to **speculative load ldf.s**
 - “Speculative” means advanced past some unknown branch
 - Processor keeps exception bit with register **f8**
 - Inserted insn **chk.s** checks exception bit
 - If exception, jump to yet more repair code (arghhh...)
 - Undo instructions after speculative load
- IA-64 also contains **ldf.sa**



Non-Biased Branches: Use Predication

- Second trace scheduling construct: **hyperblock**

- Use when branch is not highly biased
- Fuse all four blocks: A,B,C,D
- Use **predication** to execute insns in B and C
- Schedule



↓ **Using Predication**

```
0: ldf Y(r1) → f2
1: fspne f2 → p1
2: ldf.p p1, W(r1) → f2
4: stf.np p1, f0 → Y(r1)
5: ldf X(r1) → f4
6: mul f4, f2 → f6
7: stf f6 → Z(r1)
```



Predication

- Conventional control
 - Conditionally executed insns also conditionally fetched
- **Predication**
 - Conditionally executed insns unconditionally fetched
 - **Full predication** (ARM, IA-64)
 - Can tag every insn with predicate, but extra bits in instruction
 - **Conditional moves** (Alpha, IA-32)
 - Construct appearance of full predication from one primitive
`cmoveq r1,r2,r3                    // if (r1==0) r3=r2;`
 - May require some code duplication to achieve desired effect
 - + Only good way of adding predication to an existing ISA
- **If-conversion**: replacing control with predication
 - + Good if branch is unpredictable (save mis-prediction)
 - But more instructions fetched and “executed”



ISA Support for Predication

```
0: ldf Y(r1), f2
1: fspne f2, p1
2: ldf.p p1, W(r1), f2
4: stf.np p1, f0, Y(r1)
5: ldf X(r1), f4
6: mulf f4, f2, f6
7: stf f6, Z(r1)
```

- IA-64: change branch 1 to **set-predicate insn fspne**
- Change insns 2 and 4 to **predicated insns**
 - **ldf.p** performs **ldf** if predicate **p1** is true
 - **stf.np** performs **stf** if predicate **p1** is false



Static Scheduling Summary

- Goal: increase scope to find more independent insns
- Loop unrolling
 - + Simple
 - Expands code size, can't handle recurrences or non-loops
- Software pipelining
 - Handles recurrences
 - Complex prologue/epilogue code
 - Requires register copies (unless rotating register file....)
- Trace scheduling
 - Superblocks and hyperblocks
 - + Works for non-loops
 - More complex, requires ISA support for speculation and predication
 - Requires nasty repair code



Multiple Issue Summary

- Problem spots
 - Wide fetch + branch prediction → trace cache?
 - N^2 dependence cross-check
 - N^2 bypass → clustering?
- Implementations
 - Statically scheduled superscalar
 - VLIW/EPIC
- What's next:
 - Finding more ILP by relaxing the in-order execution requirement